AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

S7 PROJECT FINAL REVIEW

FlexaScale

Reinforcement Learning-Based Intelligent Auto-Scaling Framework for Distributed Microservices on Kubernetes

Guided by
Dr. Lakshmi Mohan, Department of CSE

Atmakrishna K Raghavan AM.SC.U4CSE23015
Dattanand U D AM.SC.U4CSE23023
G O Ashwin Praveen AM.SC.U4CSE23027
Ghanshyam S Sunil AM.SC.U4CSE23029

Team A1 · Amrita School of Computing, Amritapuri · November 2026

Outline

1 Introduction

2 Problem Definition

3 Objectives

4 Literature Review

5 Research Gap

6 Datasets, Platforms and Tools

7 Proposed Methodology

8 Proposed Approach

9 Evaluation Setup and Criteria

10 Results and Analysis

11 Discussion and Limitations

12 Conclusion

13 Future Work

14 Team Contribution

15 Appendix A: Code and Demo

16 Project Time Line

17 Reference

Introduction

Background and motivation

1

Microservices run on Kubernetes

An online shop is many small services. Kubernetes keeps N replicas of each one running.

2

The default autoscaler is simple

The HPA scales each service on its own CPU: desired = ⌈replicas × CPU / target⌉, then waits 5 min before scaling down.

3

Simple rules leave gaps

It cannot see call dependencies or non-CPU bottlenecks, and it keeps spike capacity long after a spike.

4

Our aim

Let a learned, dependency-aware policy scale the services, without giving up the HPA's safety.

Sustainable Development Goals

SDG 9

Industry, Innovation and Infrastructure

Target 9.4: resource-efficient, reliable cloud infrastructure

SDG 12

Responsible Consumption and Production

Target 12.2: fewer idle replicas, less allocated compute

Demonstrated on a laptop cluster; production-scale savings are expected, not measured.

Problem Definition

Problem statement

Choose the replica count of every service, every 10 s, so that the application uses as few **replica-hours** as possible while end-to-end **p95 latency stays ≤ 1000 ms**, and the cluster stays safe.

Safety

Bounded actions: one bad decision must not overload the node or empty a busy service.

Exclusive control

Each service has an HPA. Exactly one controller may own a service at a time.

Graceful fallback

If telemetry fails, the controller crashes or the policy is unsure, the HPA takes over.

What we measured with the default HPA

27

replicas at the flash-sale peak, kept for 5 min after it ended

+15%

replica-hours in checkout-heavy load, for a brief CPU rise

10×

order-service latency rise while its CPU stayed at 30-50% of limit

Objectives

01 Framework

Autoscaling add-on for Kubernetes microservices; no application code changes

Evaluated by: Shop (16) and bookstore (5)

02 RL decision engine

PPO policy that learns from simulated workloads and runs on live metrics

Evaluated by: Twin validation, live inference

03 Dependency-aware

Reason over the service call graph with a graph neural network

Evaluated by: Graph vs no-graph ablation

04 Safe operation

Bounded actions, exclusive ownership, automatic fallback to the HPA

Evaluated by: Fault injection, cleanup checks

05 Fair evaluation

Compare with default and tuned HPA on latency, cost and SLO

Evaluated by: Pre-registered paired study, 95% CI

06 Deployment & monitoring

Real-time dashboard and Helm packaging

Evaluated by: Live runs, Helm lifecycle 33/33

Literature Review

Twelve primary papers, 1-6

Paper	Year	Key contribution	Dataset / setting	Evaluation metrics	Demerit / gap
Gwydion [1]	2025	RL autoscaler with dependency-aware state; cost + latency reward	Kubernetes; Redis Cluster, Online Boutique	Latency, resources vs HPA	Horizontal only; default-HPA baseline
Smart HPA / ProSmart [2]	2025	Reactive and ML-predictive resource-efficient HPA	Kubernetes microservice benchmark	Utilisation, over/under-provisioning	No RL; dependencies not modelled
RL autoscaling [3]	2025	PPO/DDPG, horizontal + vertical scaling	Amazon EKS, Spark TPC-DS	Cost, performance	Long training; no dependencies
AdaptScale [4]	2020	Adaptive prediction + greedy heuristic for multiple SLAs	Simulation platform	SLA satisfaction, resources	Cloud data scaling; greedy
KubCG [5]	2021	Scheduler using execution history	Heterogeneous CPU-GPU cluster	Job completion time	Placement, not autoscaling
Rossi et al. [6]	2019	Model-based RL for horizontal + vertical elasticity	Simulation; Docker Swarm	Performance, resource, adaptation cost	Single app; no dependencies

Literature Review

Twelve primary papers, 7-12

Paper	Year	Key contribution	Dataset / setting	Evaluation metrics	Demerit / gap
FIRM [7]	2020	SVM critical-path localisation + DDPG reprovisioning	Kubernetes; DeathStarBench, Train-Ticket	SLO violations (up to 16× fewer), CPU	Needs tracing; acts on limits
Autopilot [8]	2020	Recommenders + ML meta-algorithm at Google	Borg production fleet	Slack 23% vs 46% manual, OOMs	Needs fleet history
Sinan [9]	2021	CNN + boosted trees predict QoS risk	DeathStarBench; local and GCE	QoS met, resources	Large per-app data collection
GRAF [10]	2021	GNN latency model + joint CPU optimisation	Kubernetes benchmarks	CPU allocation, SLO	Per-app model; needs forecasts
Alibaba trace [11]	2021	Production microservice trace and analysis	>20,000 services	Topology, dynamics	No controller evaluated
DeathStarBench [12]	2019	Open-source end-to-end microservice apps	Real servers, cloud/edge	Latency, tail effects	Heavy for small hardware

Research Gap

Identified from the twelve reviewed works

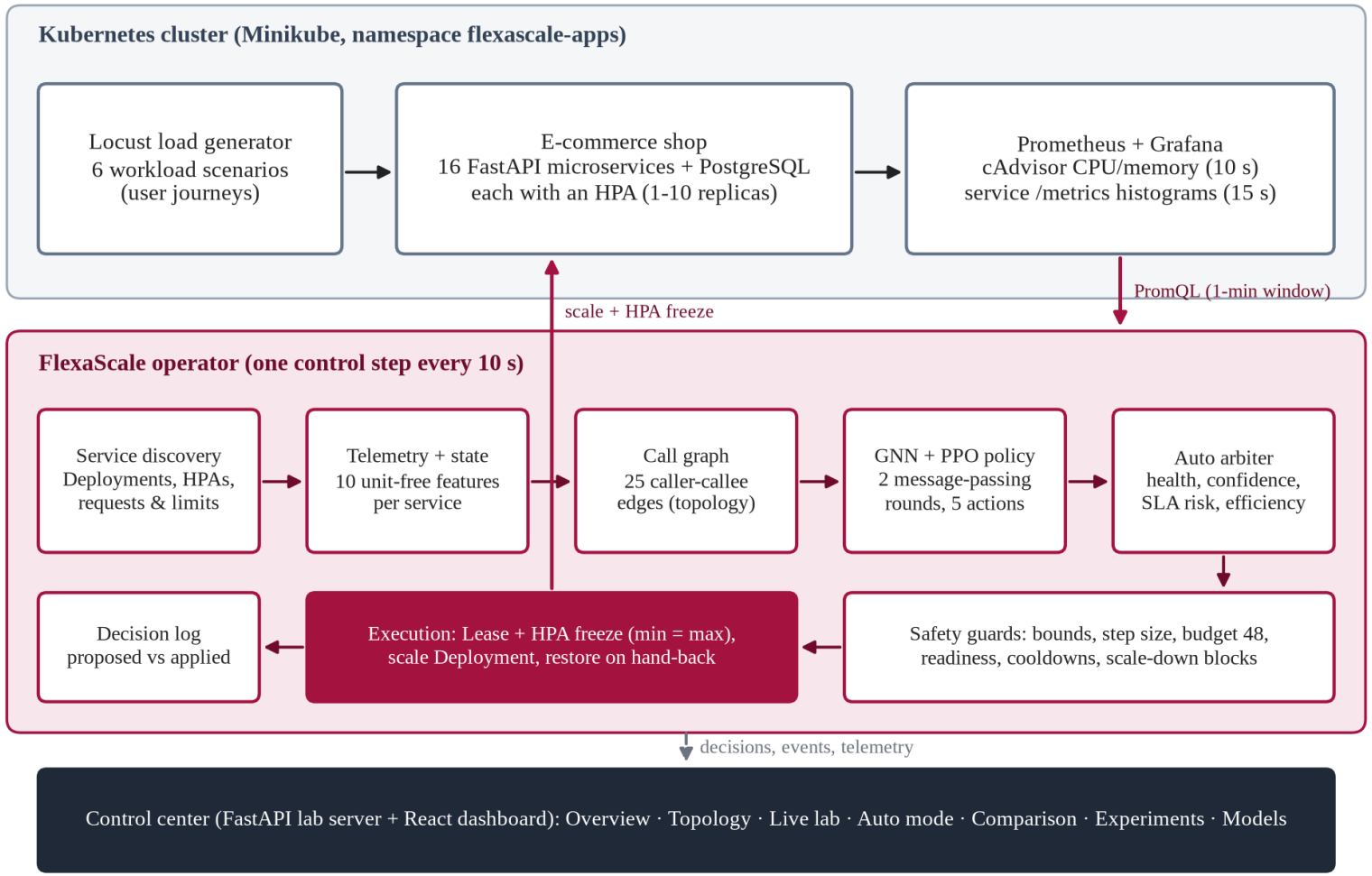
Gap in the literature	How FlexaScale addresses it
Baselines are weak Learned autoscalers are compared with the default HPA, so it is unknown how much a tuned rule would also gain.	**Compare with default and a calibrated, tuned HPA under rules fixed before the runs**
Coexistence is ignored Few works say who owns a service, or what happens when telemetry or the controller fails.	**Per-service Auto arbitration, Lease + HPA freeze, verified fallback**
Graph benefit is assumed Dependencies are used in many ways, but their benefit is rarely isolated.	**Graph vs no-graph ablation on three held-out test sets**

Datasets, Platforms, and Implementation Tools

Category	Used in FlexaScale	Purpose
Dataset	Alibaba microservice trace v2021 (7.4 GB raw → 36,355 rows, split by service); calibration runs; 289+ live runs	Train v1-v6; fit the twin; evaluate
Platform	Kubernetes v1.31 on single-node Minikube (Docker driver)	System under control
Hardware	Laptop 1: Intel i5-1340P, 16 GB, Fedora 43 · Laptop 2: Ryzen 7 6800H, RTX 3050 4 GB, Windows 11	Development, verification · GPU training, v9 evaluation
Learning	PyTorch 2.13, Stable-Baselines3 2.9 (PPO), Gymnasium 1.3	GNN + PPO policy, simulators
Monitoring	Prometheus + Grafana (kube-prometheus-stack 92.11), cAdvisor	CPU, memory, request and latency metrics
Application & load	16 FastAPI microservices, PostgreSQL 16, React frontend; Locust 2.46	Test application and workloads
Control center	FastAPI lab server; React 18, TypeScript, Vite, Tailwind	Dashboard and experiment API
Packaging & testing	Helm chart 0.5.1, operator image 0.6.0; pytest (373 tests)	Installation; verification

Proposed Methodology

Architecture: an add-on that controls the application from outside



Novelty

Graph policy

GNN message passing over the real call graph, PPO actor

Auto mode

Per-service choice between FlexaScale and HPA, with reasons

Exclusive ownership

Lease + HPA freeze; never two controllers on one service

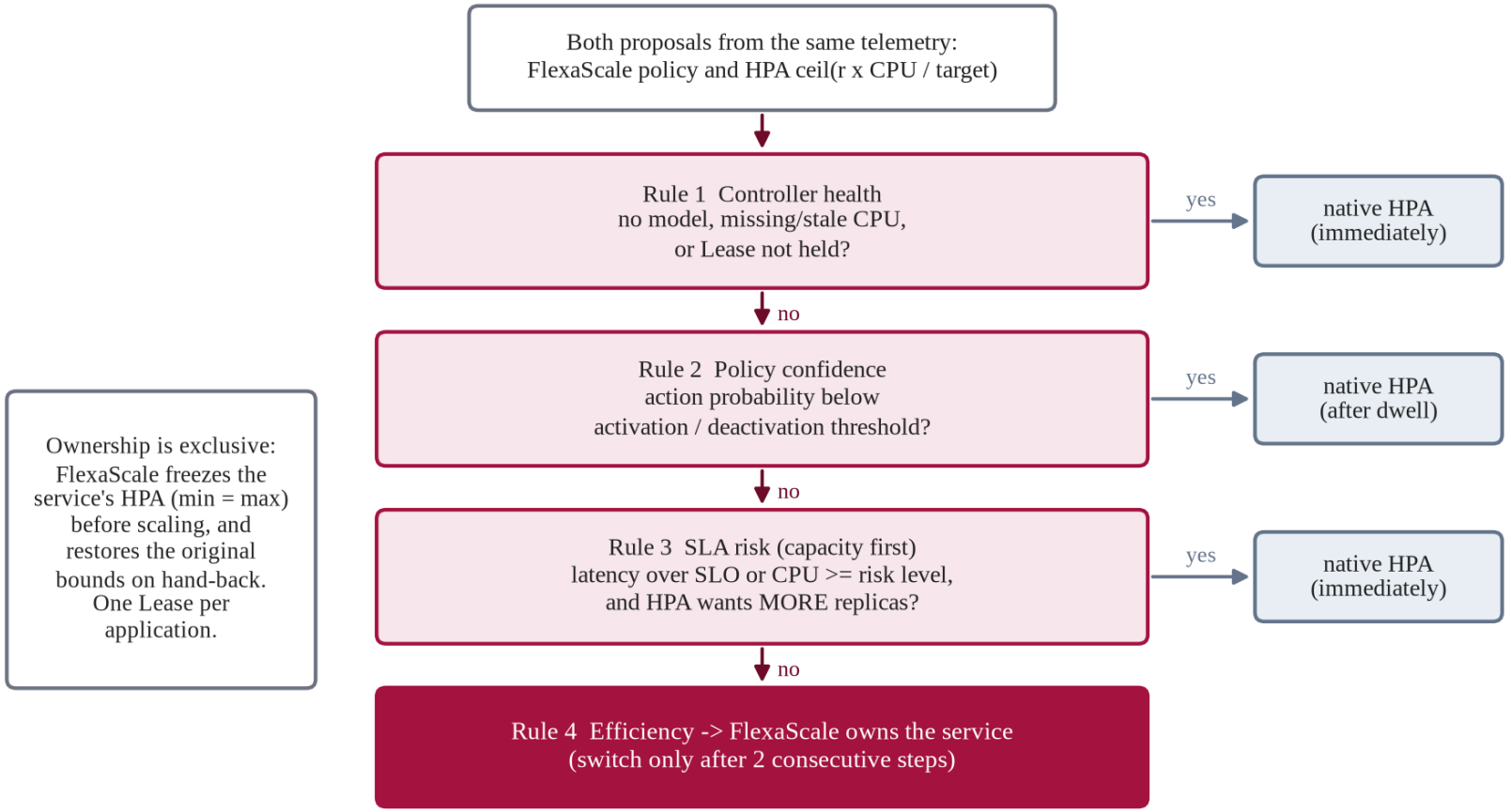
Verified fallback

Telemetry loss, crash or Lease loss → HPA

AMRITA VISHWA VIDYAPEETHAM

Proposed Methodology

Auto mode and the safety layer



Safety guards

- Replicas within 1-10
- At most +3 / −1 per step
- Budget: 48 replicas in total
- Cooldowns 20 s up, 30 s down
- No scale-up while pods start
- No scale-down at CPU ≥ 400% of request
- Stale telemetry → HPA

Proposed Approach

Algorithm, training and testing, hyperparameters

Algorithm (every 10 s)

1. Discover services and HPAs
2. Query Prometheus (1-min window)
3. Build 10 features per service
4. GNN + PPO → action −1…+3
5. Auto rules choose the owner
6. Guards approve or reduce
7. Freeze HPA, scale, log

Training and testing

- Fit an application twin to live runs (p95 error 8.9%)
- Warm start: imitate limit_target(0.6/0.3)
- PPO fine-tuning: 2M steps per run, seeds 1-2
- Select on 40 validation workloads; gate vs v7
- Test once on T1-T3 held-out sets
- Live: pre-registered paired evaluation

Hyperparameters

Parameter	Value
Learning rate	3e-4 → 1e-4
Clip range	0.2 → 0.1
Discount γ / GAE λ	0.97 / 0.95
Rollout / batch / epochs	64 / 512 / 10
Entropy coefficient	0.01
Parallel agents	4 sims × 20
Hidden / MP rounds	64 / 2
Steps per run	2,000,000

→ *after warm start; v9 trained on RTX 3050 (1.79× CPU)*

Evaluation Setup and Criteria

Setup

- **Workloads:** 6 Locust scenarios (steady, ramp, flash sale, checkout heavy, traffic drop, payment degradation); 25 users, 300 s
- **Baselines:** default HPA (70%, 300 s window) and tuned HPA (350%, 60 s, chosen by a fixed calibration rule)
- **Candidates:** FlexaScale RL and FlexaScale Auto
- **Design:** 12 blocks, 4 arms back to back in seeded order; rules fixed before the first run
- **Validity:** settled start, cleanup safe, AC power, host load within idle + 2 cores → 46 of 53 runs valid

Metrics and formulas

Replica-hours	RH = Σt Σi ready_i(t) · Δt / 3600
Paired difference	Δ% = 100 · (candidate − reference) / reference
95% confidence interval	mean Δ ± t(0.975, n−1) · s / √n
SLO violation	seconds of 10 s windows with p95 > 1000 ms
Estimated cost	vCPU-h × $0.04048 + GiB-h × $0.004445

AMRITA VISHWA VIDYAPEETHAM

Results and Analysis

End-to-end verification on the live cluster

Run	Service	Selected owner	Decision reason	Policy invoked	Action applied	Actual replicas
Auto, default	order	FlexaScale	efficiency	ppo_graph	1→2 at +289 s (desired +5.9 s, ready +15.9 s)	1-2 / 2 ready
Auto, SLA risk	order	FlexaScale 16 · HPA 15	efficiency 19 · sla_risk 12	ppo_graph	1→2 at +29 s; 3→2, 2→1 later	1-3 / 3 ready
Auto, SLA risk	catalog	FlexaScale 18 · HPA 13	efficiency 21 · sla_risk 10	ppo_graph	3→2 at +229 s; 2→1 at +259 s	1-3 / 3 ready
Auto, SLA risk	11 others	FlexaScale 30 · HPA 1	efficiency 31	ppo_graph	none needed	1 / 1

21 / 31

steps with mixed ownership (SLA-risk run)

16 → HPA

in the same step when Prometheus was cut

+1 s

watchdog restored all HPAs after SIGKILL

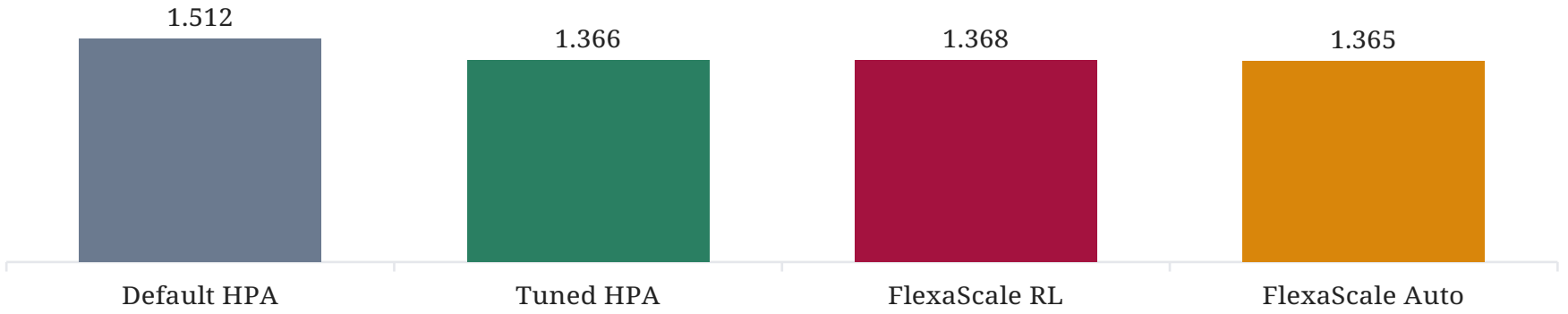
0

exclusivity violations in 9 runs

Results and Analysis

Pre-registered evaluation (46 valid runs, second laptop)

Comparison	Pairs	Replica-hours Δ [95% CI]	p95 Δ [95% CI]	Verdict
Tuned HPA vs default HPA	12	−9.0% [−13.9, −4.1]	−10.7% [−27.5, +6.1]	Beats on cost
FlexaScale RL vs default HPA	11	−7.3% [−11.5, −3.2]	+53.5% [−49.4, +156.4]	On par
FlexaScale Auto vs default HPA	11	−7.7% [−12.3, −3.2]	+17.6% [−6.0, +41.2]	On par
FlexaScale RL vs tuned HPA	11	+0.3% [−0.2, +0.7]	+155% [−165, +475]	On par
FlexaScale Auto vs tuned HPA	11	+0.2% [−0.2, +0.5]	+41.5% [−22.6, +105.7]	On par



Key result

−7.7%

replica-hours vs default HPA (Auto), CI below 0

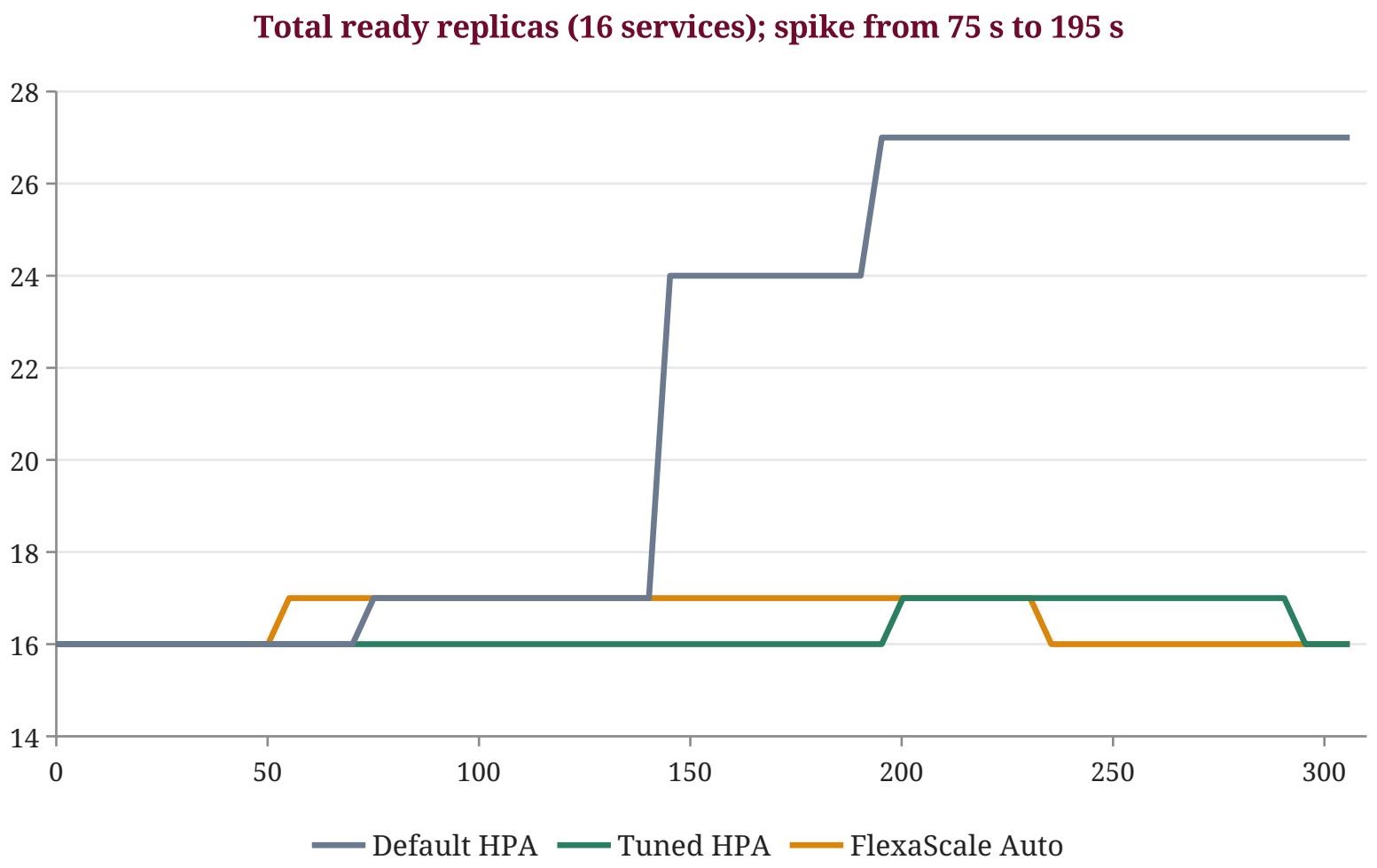
≈ 0%

vs tuned HPA in two independent studies

Mean replica-hours per arm (left). Latency checks were not met, so the verdict vs default HPA is "on par".

Results and Analysis

Visual output: flash sale, block 4 (E1)



−22.9%

replica-hours vs default HPA

−51.2%

p95 latency (410 → 200 ms)

30 s → 0 s

time above the SLO

One pair; tuned HPA reached 1.386 vs Auto 1.410 replica-hours.

Results and Analysis

Comparison with the state of the art

Aspect	Gwydion [1]	FIRM [7]	GRAF [10]	FlexaScale
Decision	Replicas (RL)	Resource limits (RL)	CPU allocation (GNN)	Replicas (GNN + PPO)
Dependencies	In the state	Critical path	GNN latency model	Message passing on call graph
Coexistence with HPA	Not reported	Not addressed	Not addressed	Per-service Auto, freeze, Lease
Failure fallback	Not reported	Not reported	Not reported	Verified (telemetry, crash, Lease)
Baseline	Default HPA	K8s autoscaling	Existing autoscalers	Default and tuned HPA
Reported gain	Better than HPA	Up to 16× fewer SLO violations	Lower CPU for same SLO	−7.7% vs default; on par vs tuned

Published gains come from other applications and mostly from default-HPA baselines, so they are not directly comparable. Against default HPA our result points the same way; against a calibrated HPA, under fixed rules, the advantage disappears.

Discussion and Limitations

What the results mean

- The system works: safe, explainable, per-service control with verified hand-back to the HPA.
- Savings vs default HPA are real (7-8%), mostly from not over-scaling spikes.
- A tuned HPA gets the same saving: at 25 users most services stay at 1 replica (floor 1.333 RH).
- The graph input showed no measurable benefit in the ablation.
- At 40 users the order service was database-bound (latency 10×, CPU 30-50%); CPU-based policies held 1 replica.

Limitations

Scale	single-node Minikube, 25-40 users, 300 s runs
One application	16-service shop (bookstore for deployment only)
Host noise	large run-to-run variance; 7 runs excluded
Calibration	thresholds and twins are per laptop
Signal	policies read mainly CPU, not latency

Conclusion

Objective-wise status: planned vs achieved

Objective	Planned	Achieved	Status
O1 Framework	Add-on for any K8s app	Shop + bookstore, no code changes	Achieved
O2 RL engine	PPO on live metrics	v1-v9; v9 GPU-trained, live inference	Achieved
O3 Dependency-aware	GNN over call graph	Implemented; ablation: no measurable gain	Achieved
O4 Safe operation	Bounded, exclusive, fallback	Verified incl. outage and SIGKILL	Achieved
O5 Fair evaluation	vs HPA: latency, cost, SLO	−7.7% vs default; on par vs tuned	Achieved
O6 Deployment	Dashboard + Helm	7-page dashboard; Helm 33/33	Achieved

373 tests pass

46 valid runs

9 verification runs

FlexaScale is a working, safe and explainable RL autoscaler. It saves resources against the default HPA and matches, but does not beat, a carefully tuned HPA.

Future Work

S8 phase

1 Latency-aware Auto

Add end-to-end and per-service latency SLOs to the state and SLA-risk rule

2 Per-cluster calibration

Derive thresholds and tuned baselines automatically on each cluster

3 Stronger baselines

Per-service tuned HPAs, custom-metric HPAs, faster metrics

4 Larger clusters

Multi-node cluster, heavier and longer workloads

5 Safe online adaptation

Fine-tune in shadow mode; deploy only after the gate

6 Publication

Paper on fair evaluation of learned autoscalers; final dissertation

Team Contribution

Main areas, from the project's Git history

GOA

G O Ashwin Praveen

AM.SC.U4CSE23027

- System integration and operator
- Auto mode and safety layer
- Experiment lab and dashboard
- Application twin, v7 training
- Live verification and Helm

DUD

Dattanand U D

AM.SC.U4CSE23023

- Minikube cluster and Helm setup
- State-vector schema
- GNN encoder
- v9 GPU training
- v9 pre-registered evaluation

AKR

Atmakrishna K Raghavan

AM.SC.U4CSE23015

- Prometheus and Grafana stack
- Metrics client and schema validation
- Confidence proxy
- HPA vs RL comparison

GSS

Ghanshyam S Sunil

AM.SC.U4CSE23029

- Demo microservices
- PPO actor-critic training pipeline

Appendix A: Code and Demo

Repository, commit and how to run

Scan for the repository

Repository

```
https://github.com/ashwinpraveengo/flexascale
branch main · commit d8416a2
```

Run the demo

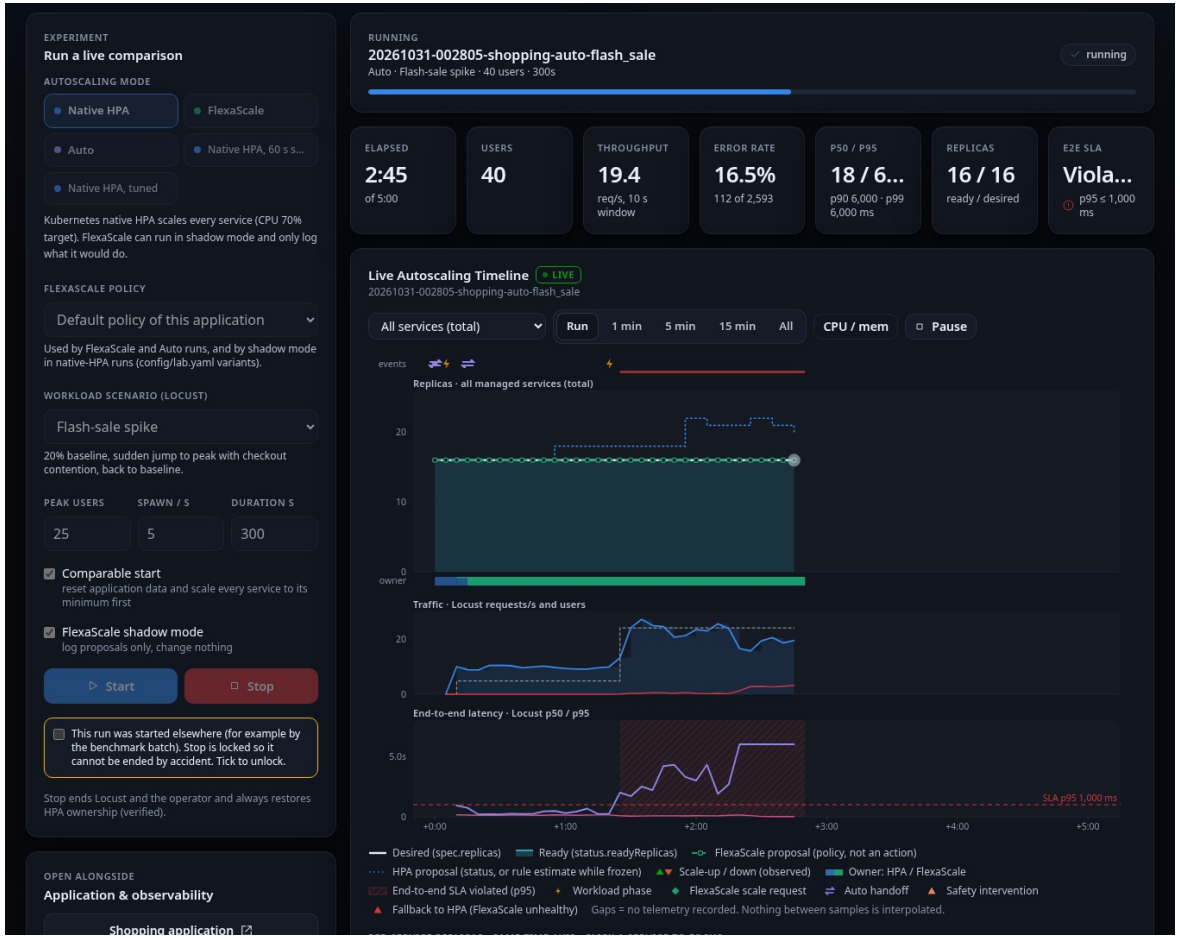
```
./fsctl start      # cluster, monitoring, shop, dashboard
open http://127.0.0.1:8090/  → Live lab → Auto → Start
./fsctl stop       # restore every HPA, verify no Lease
```

Results: results/RESULTS.md · Verification: results/verification/ · Tests: pytest (373)

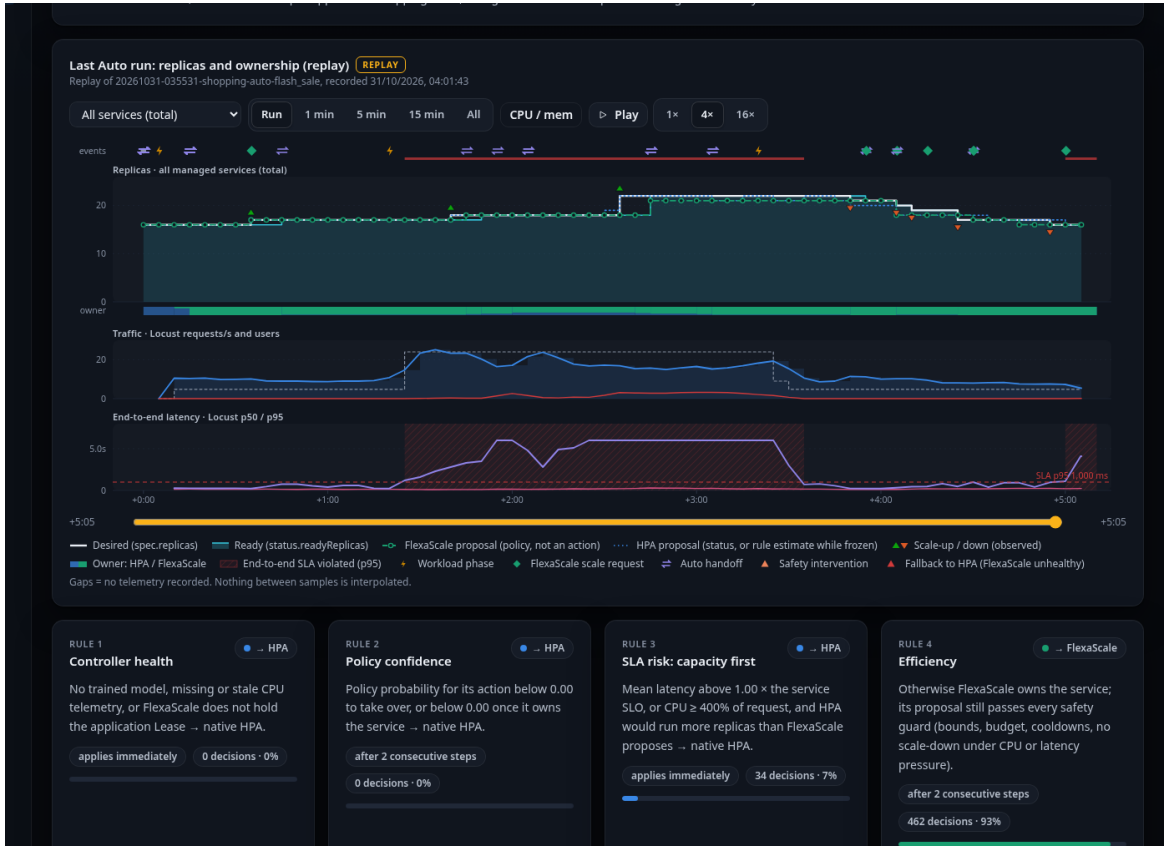
On the Fedora laptop choose the v7 policy; v9 was fitted to the second laptop.

Appendix A: Code and Demo

Demo screenshots from the control center



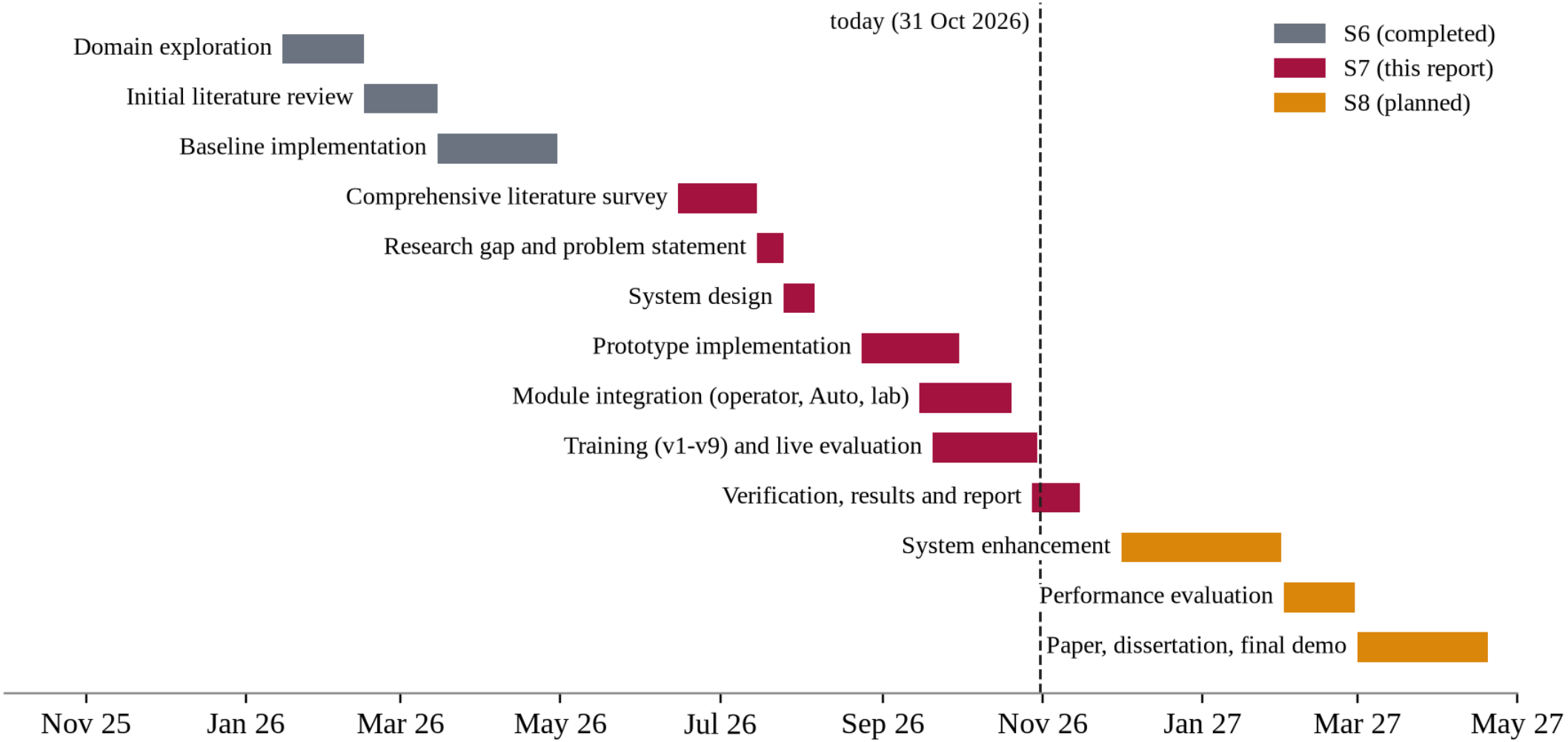
Live lab during an Auto-mode flash sale



Auto mode page: owner band, hand-offs and rule usage (SLA risk 7%, efficiency 93%)

AMRITA VISHWA VIDYAPEETHAM

Project Time Line



Reference

[1] J. Santos et al., “Gwydion: Efficient auto-scaling for complex containerized applications in Kubernetes through RL,” JNCA, vol. 234, 2025.

[2] H. Ahmad et al., “Towards resource-efficient reactive and proactive auto-scaling for microservice architectures,” JSS, vol. 225, 2025.

[3] V. Pandey, “RL-based autoscaling for cost and performance optimization in Kubernetes clusters,” 2025.

[4] Y. Shi et al., “AdaptScale: An adaptive data scaling controller …,” FGCS, vol. 105, pp. 814-823, 2020.

[5] G. El Haj Ahmed et al., “KubCG: A dynamic Kubernetes scheduler for heterogeneous clusters,” SPE, vol. 51, no. 2, 2021.

[6] F. Rossi et al., “Horizontal and vertical scaling of container-based applications using RL,” IEEE CLOUD, 2019.

[7] H. Qiu et al., “FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices,” OSDI, 2020.

[8] K. Rzadca et al., “Autopilot: Workload autoscaling at Google,” EuroSys, 2020.

[9] Y. Zhang et al., “Sinan: ML-based and QoS-aware resource management for cloud microservices,” ASPLOS, 2021.

[10] J. Park et al., “GRAF: A GNN based proactive resource allocation framework for SLO-oriented microservices,” CoNEXT, 2021.

[11] S. Luo et al., “Characterizing microservice dependency and performance: Alibaba trace analysis,” SoCC, 2021.

[12] Y. Gan et al., “An open-source benchmark suite for microservices …,” ASPLOS, 2019.

[13] J. Schulman et al., “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.

[15] A. Raffin et al., “Stable-Baselines3: Reliable RL implementations,” JMLR, vol. 22, 2021.

[19] Kubernetes Authors, “Horizontal Pod Autoscaling,” kubernetes.io documentation, 2026.